

Model Fine-Tuning

When prompting isn't enough — pretraining vs. fine-tuning, instruction tuning, SFT, RLHF, DPO, LoRA, QLoRA, adapters, dataset preparation, and fine-tuning evaluation.

DIFFICULTY LEVEL

Advanced

ESTIMATED READING TIME

55-70 minutes

ESTIMATED PRACTICE TIME

90-120 minutes

PREREQUISITES

Modules 01-20

LEARNING OUTCOMES

By the end of this module, you will be able to: explain how fine-tuning differs from pretraining and from prompting; describe SFT, RLHF, and DPO conceptually; understand why LoRA/QLoRA and adapters make fine-tuning more efficient; prepare a fine-tuning dataset correctly; and evaluate whether fine-tuning actually improved a model for your task.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Fine-Tuning Basics
3.2	Pretraining vs. Fine-Tuning
3.3	Instruction Tuning
3.4	SFT (Supervised Fine-Tuning)
3.5	RLHF
3.6	DPO
3.7	LoRA & QLoRA
3.8	Adapters
3.9	Dataset Preparation
3.10	Fine-Tuning Evaluation
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

📖 What is this topic?

Every technique in this course so far has worked with a frozen, unchanged model — shaping behavior entirely through what you send it (prompts, context, tools). Fine-tuning is different: it actually adjusts the model's own weights, teaching it new patterns of behavior directly.

🎯 Why is it important?

Fine-tuning is a genuinely different lever from everything in Modules 02-09 — instead of shaping behavior per-request through the prompt, you shape it once, into the model itself. Knowing when this tradeoff is actually worth it (versus better prompting or RAG) is a real engineering decision.



🌐 Why should AI Engineers learn it?

Most teams should try prompting, context engineering (Module 05), and RAG (Module 07) exhaustively before fine-tuning — but knowing what fine-tuning actually is (and its real costs) is what lets you make that call correctly instead of reaching for it too early or dismissing it when it's genuinely needed.

🌍 Where is it used in the real world?

Teams fine-tune for very specific, narrow behaviors that prompting struggles to reliably enforce — a highly specific output format at massive scale, a distinctive brand voice, or domain-specialized reasoning (legal, medical) that general prompting can't fully capture.

🔔 Why is it relevant today?

Parameter-efficient techniques (LoRA, QLoRA, Section 3.7) have made fine-tuning far more accessible than it was even a couple of years ago — no longer requiring the massive infrastructure that full fine-tuning demands, especially for open-weight models (Module 11).

Note

This module is deliberately conceptual, not a hands-on fine-tuning tutorial — actual fine-tuning workflows vary significantly by provider and framework. The goal is understanding what each technique does and when to reach for it, not memorizing a specific implementation.

2. Before You Begin

RECAP

Model capabilities & limitations (Module 01, 3.13) — fine-tuning is one way to shift these limitations, but not the only or always the best one.

RECAP

Open-weight models (Module 11, 3.4) — fine-tuning is generally only possible with open-weight models or specific providers' managed fine-tuning offerings, not arbitrary closed APIs.



Warning

Fine-tuning is a significant investment of data, compute, and expertise. Before reaching for it, genuinely exhaust prompting (Modules 02-03), context engineering (Module 05), and RAG (Module 07) — many problems people assume need fine-tuning are actually solvable with much less effort.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

🔧 3.1 Fine-Tuning Basics

DEFINITION

Fine-tuning is further training an already-trained model on a smaller, specific dataset, adjusting its weights to shift its behavior toward that dataset's patterns.

SIMPLE EXPLANATION

Instead of shaping the model's output through what you put in the prompt (everything in Modules 02-09), fine-tuning changes the model itself — the resulting behavior persists across every future call, without needing to repeat instructions in every prompt.

💡 *Like the difference between giving someone detailed instructions every single time you ask them to do a task (prompting) versus actually training them once so the skill becomes second nature (fine-tuning) — after training, they no longer need the detailed instructions repeated.*

EVERYDAY EXAMPLE

Instead of including 10 few-shot examples (Module 03, 3.3) in every single prompt to enforce a specific output style, fine-tuning bakes that style directly into the model, so every future call produces it by default.

WHY IT MATTERS

Fine-tuning trades upfront data/compute investment for reduced per-request prompt complexity and (often) more reliable, consistent behavior at scale.

COMMON MISTAKES

Reaching for fine-tuning as a first resort for problems that better prompting or few-shot examples (Module 03) would have solved more cheaply.

BEST PRACTICES

Exhaust prompting, context engineering, and RAG first; fine-tune only when you have clear evidence (via evaluation, Module 16) that those approaches have a real, persistent ceiling for your specific task.

SUMMARY

Fine-tuning shapes model behavior by adjusting weights directly, trading upfront investment for reduced per-request prompt complexity — a different lever from everything else in this course.

🏗️ 3.2 Pretraining vs. Fine-Tuning

DEFINITION

Pretraining is the initial, massive-scale training process that creates a base LLM's general language and reasoning capability; fine-tuning is a much smaller, subsequent training step that specializes an already-pretrained model.

SIMPLE EXPLANATION

Pretraining requires enormous datasets (much of the internet) and compute, producing a general-purpose model; fine-tuning requires a tiny fraction of that — often just thousands of examples — to specialize a model that already has broad general capability.

💡 *Like the difference between a person's entire general education (pretraining — years, broad, foundational) and a focused professional certification course (fine-tuning — weeks, narrow, building directly on that foundation).*

EVERYDAY EXAMPLE

Claude's base capability (reasoning, general knowledge, language understanding) comes from pretraining; a hypothetical fine-tuned variant specialized for legal document review would build a narrow specialization on top of that same foundation.

WHY IT MATTERS

Understanding this distinction clarifies why fine-tuning is so much more accessible than building a model from scratch — you're refining, not recreating, the underlying general capability.

COMMON MISTAKES

Confusing fine-tuning's scale and cost with pretraining's — they are vastly different orders of magnitude in data, compute, and expertise required.

BEST PRACTICES

Think of fine-tuning as specialization on top of pretraining's broad foundation, not as an alternative to it — a fine-tuned model still relies heavily on everything it learned during pretraining.

SUMMARY

Pretraining builds broad, general capability at massive scale; fine-tuning specializes that capability with far less data and compute — a specialization step, not a replacement.

3.3 Instruction Tuning

DEFINITION

Instruction tuning is a specific type of fine-tuning that trains a base pretrained model to follow instructions and hold conversations, rather than just predicting plausible continuations of text.

SIMPLE EXPLANATION

A raw pretrained model (Section 3.2) is good at continuing text patterns but isn't naturally good at following an instruction like "summarize this in 3 bullet points" — instruction tuning is what teaches that specific behavior, and is why modern assistant-style LLMs feel conversational and instructable rather than just text-completing.

 *Like the difference between someone who's extremely well-read (pretraining) and someone who's also been specifically trained in customer service to actually respond helpfully to requests (instruction tuning) — the underlying knowledge is the same, but the behavior is very different.*

EVERYDAY EXAMPLE

The reason Claude responds helpfully to "explain quantum computing simply" instead of just continuing the sentence with more similar-sounding text is instruction tuning — it's been trained specifically to interpret and follow such requests.

WHY IT MATTERS

Instruction tuning is what makes every technique in Modules 02-03 (giving instructions, expecting them to be followed) work at all — a raw, non-instruction-tuned base model wouldn't reliably follow prompts the way this entire course assumes.

COMMON MISTAKES

Assuming all fine-tuning is instruction tuning — it's actually one specific, common application of the broader fine-tuning concept.

BEST PRACTICES

Understand that the models you've used throughout this course are already instruction-tuned — your own fine-tuning (if needed) typically specializes further from that instruction-following baseline, not from scratch.

SUMMARY

Instruction tuning teaches a base model to follow instructions and converse — the foundational fine-tuning step that makes every prompting technique in this course possible in the first place.

3.4 SFT (Supervised Fine-Tuning)

DEFINITION

SFT (Supervised Fine-Tuning) is training a model on a labeled dataset of input-output example pairs, directly teaching it "given this input, produce output like this."

SIMPLE EXPLANATION

This is the most straightforward fine-tuning method — you gather many examples of the exact behavior you want (a specific tone, a specific format, domain-specific reasoning) and train the model to reproduce that pattern, similar in spirit to Module 03's few-shot examples but baked in permanently instead of repeated per-prompt.

 Like teaching by direct demonstration — showing many worked examples of "here's the problem, here's exactly how to solve it," and having the learner internalize the pattern from repeated exposure.

EVERYDAY EXAMPLE

Fine-tuning a model on 5,000 examples of customer support tickets paired with ideal responses, teaching it your company's specific tone and resolution style directly.

WHY IT MATTERS

SFT is the most common, accessible starting point for fine-tuning — conceptually simple and directly tied to concrete example data you can gather and review.

COMMON MISTAKES

Using inconsistent or low-quality example pairs, which the model will faithfully learn to reproduce — SFT output quality is capped by input dataset quality.

BEST PRACTICES

Curate SFT datasets as carefully as Module 16's golden datasets — consistency and quality of examples directly determines the fine-tuned model's behavior.

SUMMARY

SFT trains directly on labeled input-output pairs — simple, accessible, and only as good as the quality and consistency of its training examples.

3.5 RLHF

DEFINITION

RLHF (Reinforcement Learning from Human Feedback) is a training technique where a model learns from human preference judgments (comparing pairs of outputs, "which is better") rather than from fixed labeled examples alone.

SIMPLE EXPLANATION

Where SFT (Section 3.4) teaches from direct examples, RLHF teaches from comparative feedback — humans rank or compare multiple model outputs, and the model is trained to produce outputs more like the preferred ones, refining behavior in ways that are hard to specify with single fixed examples alone (like nuanced helpfulness or tone).

 Like coaching through comparative feedback ("this draft is better than that one, here's roughly why") rather than a single "correct" example — useful for refining qualities that are easier to judge comparatively than to specify as one fixed right answer.

EVERYDAY EXAMPLE

Anthropic's own safety and helpfulness training for Claude uses human feedback signals

(conceptually related to RLHF) to shape behavior around nuanced qualities like being genuinely helpful without being harmful — qualities hard to fully capture in a fixed example dataset.

WHY IT MATTERS

RLHF is largely responsible for the "feels genuinely helpful and aligned with human preferences" quality of modern assistant models — a dimension SFT alone struggles to fully capture.

COMMON MISTAKES

Confusing RLHF with SFT — RLHF specifically involves a comparative/preference feedback signal and a more complex training process, not just labeled examples.

BEST PRACTICES

Understand RLHF conceptually as "learning from comparative preference," even if you never implement it yourself — most engineers consume RLHF-trained models rather than building the RLHF pipeline from scratch.

SUMMARY

RLHF trains from human comparative preferences rather than fixed examples — largely responsible for the nuanced helpfulness of modern assistant models like Claude.

⚖️ 3.6 DPO

DEFINITION

DPO (Direct Preference Optimization) is a more recent, simpler alternative to RLHF that trains directly on preference pairs (chosen vs. rejected outputs) without RLHF's more complex reinforcement learning machinery.

SIMPLE EXPLANATION

DPO achieves a similar goal to RLHF (Section 3.5) — learning from comparative preferences — but with a more direct, simpler training objective, making it more accessible and stable to implement than full RLHF for many practitioners.

💡 *Like reaching the same destination via a more direct route — DPO and RLHF both use "which output is preferred" feedback, but DPO's path from that feedback to a trained model is more straightforward.*

EVERYDAY EXAMPLE

A team fine-tuning an open-weight model (Module 11) using DPO on a dataset of "preferred vs. rejected" response pairs specific to their product's desired tone, without needing RLHF's full reinforcement learning setup.

WHY IT MATTERS

DPO has made preference-based fine-tuning meaningfully more accessible to teams without deep reinforcement learning expertise, compared to full RLHF.

COMMON MISTAKES

Assuming DPO and RLHF are interchangeable in every detail — they share a goal (learning from preferences) but differ in their underlying training mechanics and complexity.

BEST PRACTICES

For teams doing their own preference-based fine-tuning on open-weight models, DPO is generally a more approachable starting point than implementing full RLHF from scratch.

SUMMARY

DPO offers a simpler, more direct path to preference-based fine-tuning than full RLHF — a more accessible option for teams without deep RL infrastructure.

🦄 3.7 LoRA & QLoRA

DEFINITION

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique that trains a small number of additional parameters instead of the model's full weight set; QLoRA combines LoRA with quantization (Module 22) to further reduce the memory required.

SIMPLE EXPLANATION

Full fine-tuning updates every one of a model's (often billions of) parameters — extremely resource-intensive. LoRA instead freezes the original model and trains a much smaller set of additional parameters that adjust its behavior, achieving similar results at a fraction of the compute and memory cost.

💡 *Like adding a small, targeted correction lens in front of an existing camera instead of redesigning the entire camera from scratch — you achieve a meaningfully different result by modifying a small, focused component rather than rebuilding everything.*

EVERYDAY EXAMPLE

Fine-tuning an open-weight model (Module 11, 3.4) on a single consumer GPU using QLoRA, a task that would be completely impractical with full fine-tuning's memory requirements.

WHY IT MATTERS

LoRA and QLoRA are largely why fine-tuning has become accessible to individual developers and smaller teams — they dramatically lower the compute and memory barrier compared to full fine-tuning.

COMMON MISTAKES

Assuming LoRA-based fine-tuning always matches full fine-tuning's quality in every case — it's usually very close for many tasks, but not strictly guaranteed identical.

BEST PRACTICES

Default to LoRA/QLoRA for most practical fine-tuning needs given the dramatically lower resource requirements; reserve full fine-tuning for cases with clear evidence it's specifically needed.

SUMMARY

LoRA and QLoRA make fine-tuning dramatically more accessible by training a small additional parameter set instead of the full model — the default approach for most practical fine-tuning today.

3.8 Adapters

DEFINITION

Adapters are small, swappable trained modules (LoRA, Section 3.7, is one specific type) that can be attached to or detached from a base model to add specialized behavior without modifying the base model itself.

SIMPLE EXPLANATION

Because techniques like LoRA train a small, separate set of parameters rather than modifying the base model directly, that trained addition can be packaged as a portable "adapter" — swappable in and out, or even combined with different adapters for different purposes, without needing separate full copies of the entire base model.

💡 *Like camera lens attachments — the camera body (base model) stays the same, and you swap different lenses (adapters) on and off depending on what you need to shoot, without needing a separate camera for each lens.*

EVERYDAY EXAMPLE

A company maintaining one base open-weight model plus several small adapters — one for customer support tone, one for internal code review style — swapping adapters per use case instead of hosting several full fine-tuned model copies.

WHY IT MATTERS

Adapters dramatically reduce storage and deployment complexity compared to maintaining multiple full fine-tuned model copies for different specialized use cases.

COMMON MISTAKES

Not realizing adapter-based approaches allow reusing a single base model deployment across many specialized behaviors, and instead deploying multiple redundant full model copies.

BEST PRACTICES

Use adapters when you need multiple specialized behaviors from a related base model — far more storage/deployment efficient than maintaining separate fully fine-tuned models.

SUMMARY

Adapters package fine-tuned specialization as small, swappable modules on a shared base model — efficient for maintaining multiple specialized behaviors at once.

3.9 Dataset Preparation

DEFINITION

Dataset preparation is collecting, cleaning, formatting, and validating the training data used for fine-tuning — arguably the single most impactful step in the entire fine-tuning process.

SIMPLE EXPLANATION

This closely mirrors Module 16's golden dataset principles, but for training data rather than evaluation data — quality, consistency, and representativeness of the fine-tuning dataset directly determines what behavior the model actually learns.

 *Like the ingredients going into a recipe — even perfect cooking technique (the fine-tuning algorithm) can't overcome low-quality, inconsistent, or mismatched ingredients (the dataset).*

EVERYDAY EXAMPLE

Before fine-tuning a customer-support-tone model (Section 3.4), carefully reviewing and cleaning 5,000 raw support tickets to remove inconsistent, off-brand, or genuinely poor-quality example responses that would otherwise get learned as the target pattern.

WHY IT MATTERS

A fine-tuned model faithfully learns whatever patterns exist in its training data — including any inconsistencies, biases, or quality problems, which then get baked permanently into the model's behavior.

COMMON MISTAKES

Rushing dataset preparation to get to the "actual" fine-tuning step faster, when dataset quality is usually the single biggest determinant of the final result.

BEST PRACTICES

Invest as much (or more) care in dataset preparation as in Module 16's golden dataset construction; review for consistency, remove low-quality examples, and ensure genuine representativeness of the target behavior.

SUMMARY

Dataset preparation determines what a fine-tuned model actually learns — the highest-leverage, most easily under-invested step in the entire process.

3.10 Fine-Tuning Evaluation

DEFINITION

Fine-tuning evaluation is applying Module 16's evaluation methodology specifically to compare a fine-tuned model against its base model (or a prompting-only alternative) to confirm the fine-tuning actually improved the target behavior.

SIMPLE EXPLANATION

Fine-tuning is expensive enough that "did it actually work, and was it worth it compared to alternatives" deserves the same rigor as any other evaluation (Module 16) — including checking for regressions in capabilities the fine-tuning wasn't targeting.

💡 *Like formally re-testing a car after a major engine modification — not just confirming the specific improvement you were after, but checking that nothing else about the car's performance quietly got worse in the process.*

EVERYDAY EXAMPLE

Benchmarking (Module 16, 3.5) a fine-tuned customer support model against the base model plus a well-crafted system prompt, on the same golden dataset, to confirm fine-tuning was actually worth its cost.

WHY IT MATTERS

Fine-tuning can sometimes cause "catastrophic forgetting" — improving the targeted behavior while degrading other general capabilities — evaluation is what catches this before deployment.

COMMON MISTAKES

Only checking whether the fine-tuned model improved on the specific targeted behavior, without checking for regressions elsewhere.

BEST PRACTICES

Evaluate a fine-tuned model on both the targeted improvement and a broader general-capability test set, benchmarked directly against the base model and against a strong prompting-only alternative.

SUMMARY

Fine-tuning evaluation confirms the investment actually paid off and didn't quietly degrade other capabilities — applying Module 16's full rigor to a fine-tuning decision.

4. Visual Learning

The Training Pipeline



Fine-Tuning Method Family

Fine-Tuning Methods

—

Example-based

SFT — fixed input-output pairs

—

Preference-based

RLHF, DPO — comparative feedback

—

Efficiency Techniques

LoRA, QLoRA, Adapters — apply to any of the above

When to Reach for Each Approach

Approach	Best For	Relative Cost
Prompting (Modules 02-03)	Most tasks, fastest iteration	Lowest
RAG (Module 07)	Knowledge grounding, current facts	Low-Medium
LoRA/QLoRA fine-tuning	Narrow, persistent behavior shift	Medium
Full fine-tuning	Deep specialization, rare need	Highest

5. Real World Applications

Product	How these fundamentals show up
Consumer assistant models (like Claude)	Instruction tuning + RLHF/preference training shape helpful, aligned behavior.
Domain-specialized enterprise models	SFT or LoRA fine-tuning on domain-specific data (legal, medical, financial).
Multi-tenant AI platforms	Adapters let one base model serve many customers' specialized behaviors efficiently.
Open-weight model deployments	QLoRA enables fine-tuning on limited hardware, making customization accessible.

6. Practical Examples

Beginner: Prompting vs. fine-tuning decision

For a task requiring a very specific, consistent output format at massive scale, argue for and against choosing fine-tuning over few-shot prompting (Module 03).

Beginner: Identifying SFT data

Sketch what a small SFT dataset (5-10 example pairs) might look like for teaching a model a specific customer support tone.

Intermediate: RLHF vs. DPO framing

Explain, in your own words, why DPO is often described as a "simpler" alternative to RLHF, based on what each method requires.

Advanced: Designing a fine-tuning evaluation

Design an evaluation plan (following Module 16's principles) for comparing a hypothetical fine-tuned model against its base model plus prompting — what would you measure, and on what dataset?

7. Code Examples

🔗Python — SFT dataset formatting (conceptual pattern)

```
# This shows the SHAPE of an SFT dataset, not a full training pipeline
# (actual fine-tuning workflows are provider/framework-specific)
import json

# Each example: an input paired with the exact desired output
sft_examples = [
    {
        "input": "Customer: My order hasn't arrived and it's been 2 weeks.",
        "output": "I'm really sorry for the delay — let's get this sorted. Could you share your order number so I can look into it?",
    },
    {
        "input": "Customer: This product broke after one use!",
        "output": "That's frustrating, and I completely understand. Let's get you a replacement — do you have your order number?",
    },
    # ... hundreds/thousands more, curated for tone consistency (Section 3.9)
]

# Most fine-tuning APIs expect a specific JSONL format like this:
with open("sft_dataset.jsonl", "w") as f:
    for ex in sft_examples:
        f.write(json.dumps({
            "messages": [
                {"role": "user", "content": ex["input"]},
                {"role": "assistant", "content": ex["output"]}
            ]
        }) + "\n")

print(f"Prepared {len(sft_examples)} SFT examples")
```

🔗Python — Comparing base vs. fine-tuned model (evaluation pattern)

```
# pip install anthropic
# Conceptual: evaluating whether fine-tuning was worth it (Section 3.10)
# vs. a strong prompting-only baseline, following Module 16's approach
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

golden_dataset = [
    {"input": "My package is late.", "expected_tone": "empathetic, solution-focused"},
    # ...
]

def eval_with_prompting_baseline(customer_message):
    system = "You are an empathetic, solution-focused customer support agent."
    r = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=150, system=system,
        messages=[{"role": "user", "content": customer_message}]
    )
    return r.content[0].text

# In a real comparison, you'd also call your fine-tuned model here
# and score both against the same golden dataset using Module 16's
# LLM-as-a-judge pattern, to decide if fine-tuning was worth its cost.
baseline_result = eval_with_prompting_baseline(golden_dataset[0]["input"])
print(baseline_result)
```



Tip

Before investing in any fine-tuning pipeline, build and run the "prompting baseline" evaluation shown above first — it's often surprising how far a well-crafted system prompt plus few-shot examples (Module 03) can go.

8. Common Mistakes

- **Reaching for fine-tuning before exhausting prompting, context engineering, and RAG**, when those often solve the problem more cheaply.
- **Confusing pretraining's scale with fine-tuning's** — they are vastly different orders of magnitude.
- **Using inconsistent or low-quality SFT examples**, which get faithfully learned as the target pattern.
- **Confusing RLHF and DPO as interchangeable** in every technical detail.
- **Deploying multiple full fine-tuned model copies** when adapters would serve the same specialized needs more efficiently.
- **Rushing dataset preparation**, when it's usually the single biggest determinant of fine-tuning quality.
- **Only evaluating the targeted improvement**, missing regressions in other capabilities.

9. Best Practices

- **Exhaust prompting, context engineering, and RAG first**, with clear evaluation evidence before considering fine-tuning.
- **Invest heavily in dataset preparation** — quality and consistency directly determine outcomes.
- **Default to LoRA/QLoRA** over full fine-tuning for most practical needs.
- **Use adapters** when you need multiple specialized behaviors from a shared base model.
- **Always benchmark a fine-tuned model against both its base model and a strong prompting baseline** before deploying it.
- **Check for regressions in untargeted capabilities**, not just improvement in the targeted behavior.

10. Hands-on Activities

Easy 5 exercises

1. Explain, in your own words, the difference between pretraining and fine-tuning.
2. List 3 signals that would suggest a task genuinely needs fine-tuning rather than better prompting.
3. Sketch the training pipeline diagram from Section 4 from memory.
4. Explain why LoRA/QLoRA made fine-tuning more accessible, in your own words.
5. Describe, conceptually, the difference between SFT and RLHF/DPO.

Intermediate 3 exercises

1. Build the SFT dataset formatting example from Section 7 with 10 of your own example pairs for a task of your choice.
2. Design a "prompting baseline" evaluation (Section 7's pattern) for a task you might otherwise consider fine-tuning for.
3. Research one real, publicly documented fine-tuning use case and summarize what problem it solved and why prompting alone wasn't sufficient.

Challenge 2 exercises

1. Design a full dataset preparation plan (on paper) for a hypothetical SFT fine-tuning project, including data sourcing, cleaning, and quality review steps.
2. Design a complete fine-tuning evaluation plan (Section 3.10) comparing a hypothetical fine-tuned model against both its base model and a prompting baseline, on both targeted and general-capability metrics.

11. Mini Project

Goal

Build a "should we fine-tune?" decision framework: a prompting baseline, an evaluation set, and a clear go/no-go recommendation for a hypothetical fine-tuning candidate task.

Requirements

- An Anthropic API key
- Python or JavaScript
- A specific, narrow task you might consider fine-tuning for (a specific tone, format, or domain behavior)

Step-by-Step Instructions

1. Define the target behavior precisely (e.g., "responds in our brand's specific customer support tone").
2. Build a golden dataset (Module 16, 3.2) of 15-20 cases with clear success criteria for that behavior.
3. Build the strongest possible prompting-only baseline (system prompt + few-shot examples, Module 03) and evaluate it against your golden dataset (Module 16's LLM-as-a-judge pattern).
4. Based on the score, write a recommendation: is prompting sufficient, or is there clear evidence fine-tuning would be needed?
5. If fine-tuning does seem justified, sketch what your SFT dataset (Section 7's format) would need to look like.

Expected Output

```
Task: Brand-specific customer support tone
Prompting baseline score: 4.1/5 (LLM-as-a-judge, faithfulness to brand voice)

Recommendation: Prompting baseline is strong. Fine-tuning NOT
justified at this time — the marginal gain doesn't offset the
dataset/training investment. Revisit if baseline score drops
below 3.5/5 on an expanded, harder test set.
```

Possible Improvements

- Expand the golden dataset with genuinely hard edge cases to stress-test the prompting baseline more rigorously.
- If you have access to fine-tuning infrastructure, actually run the comparison for real instead of just the baseline.
- Document this decision framework as a reusable template for future "should we fine-tune?" questions.

12. Review Questions

Multiple Choice (10)

1. Fine-tuning differs from prompting primarily by:

- A) Being cheaper always B) Adjusting the model's own weights instead of shaping behavior per-request C) Being identical to prompting D) Requiring no data

2. Pretraining compared to fine-tuning is:

- A) The same scale B) Vastly larger in data and compute C) Smaller in scale D) Not related at all

3. Instruction tuning specifically teaches a model to:

- A) Generate images B) Follow instructions and converse, rather than just continue text C) Run faster D) Use less memory

4. SFT trains a model using:

- A) Comparative preference pairs only B) Labeled input-output example pairs C) No data at all D) Random noise

5. RLHF trains a model using:

- A) Fixed labeled examples only B) Human comparative preference feedback C) No human involvement D) Pretraining data only

6. DPO is generally considered:

- A) Identical to SFT B) A simpler, more direct alternative to RLHF for preference-based training C) Unrelated to preferences D) More complex than RLHF in every way

7. LoRA achieves efficiency by:

- A) Training all model parameters B) Training a small additional parameter set instead of the full model C) Requiring more compute than full fine-tuning D) Avoiding training entirely

8. Adapters are best described as:

- A) A replacement for the base model B) Small, swappable trained modules attached to a shared base model C) A type of prompt D) Unrelated to fine-tuning

9. Dataset preparation quality:

- A) Doesn't affect fine-tuning results B) Directly determines what behavior a fine-tuned model learns C) Only matters for RLHF D) Is irrelevant to SFT

10. Fine-tuning evaluation should check for:

- A) Only the targeted improvement B) Both targeted improvement and regressions in other capabilities C) Nothing, evaluation is unnecessary D) Only cost, not quality

Identification (5)

1. _____ trains a model on labeled input-output example pairs.

2. _____ trains a model from human comparative preference feedback using reinforcement learning.

3. _____ is a simpler, more direct alternative to RLHF for preference-based training.

4. _____ is a parameter-efficient fine-tuning technique training a small additional parameter set.

5. _____ combines LoRA with quantization to further reduce memory requirements.

True or False (5)

1. Fine-tuning should generally be tried before exhausting prompting and RAG approaches.

2. A fine-tuned model faithfully learns whatever patterns exist in its training data, including flaws.

3. Adapters allow multiple specialized behaviors from one shared base model.

4. Fine-tuning evaluation should only check the targeted behavior, never other capabilities.

5. LoRA and QLoRA have made fine-tuning significantly more accessible than full fine-tuning.

Situational (5)

- 1. Your team wants perfectly consistent output formatting at massive scale. What should you try before committing to fine-tuning?**
- 2. You have limited GPU memory but want to fine-tune an open-weight model. What technique fits best?**
- 3. You need several different specialized behaviors from a similar base model without hosting many full copies. What approach fits?**
- 4. Your fine-tuned model improved on its target task but seems to have gotten worse at general reasoning. What evaluation step would have caught this earlier?**
- 5. Your SFT dataset has many inconsistent, low-quality examples. What's the most likely consequence?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-SFT, 2-RLHF, 3-DPO, 4-LoRA, 5-QLoRA

True or False: 1-False, 2-True, 3-True, 4-False, 5-True

Situational (sample reasoning):

1. Exhaust prompting techniques (few-shot examples, structured output, Module 03-04) first.
2. QLoRA — combines LoRA's efficiency with quantization for lower memory needs.
3. Adapters — swappable specialized modules on one shared base model.
4. Evaluating on a broader general-capability test set alongside the targeted metric (Section 3.10).
5. The fine-tuned model will faithfully learn and reproduce those inconsistencies and quality problems.

13. Cheat Sheet

The Pipeline

Pretraining → Instruction Tuning → RLHF/DPO → (optional) Your Fine-Tuning.

Efficiency

LoRA/QLoRA — small trained addition, not full weights.

Adapters — swappable, shared base model.

Methods

SFT — fixed examples.

RLHF — preference + RL.

DPO — simpler preference training.

Golden Rule

Exhaust prompting/RAG first. Dataset quality determines everything. Always evaluate against a baseline.

14. Key Takeaways

- Fine-tuning adjusts a model's weights directly, trading upfront investment for reduced per-request prompt complexity — a different lever from prompting.
- Pretraining builds broad general capability at massive scale; fine-tuning specializes it with far less data and compute.
- Instruction tuning is what makes models follow instructions at all — the foundation this entire course's prompting techniques rely on.
- SFT trains from fixed examples; RLHF and DPO train from comparative human preferences, with DPO being the more accessible alternative.
- LoRA and QLoRA make fine-tuning dramatically more accessible by training a small parameter addition instead of the full model.
- Adapters let one base model serve multiple specialized behaviors efficiently.
- Dataset preparation is usually the single biggest determinant of fine-tuning quality — invest accordingly.
- Always evaluate a fine-tuned model against both its base model and a strong prompting baseline before deploying it.

15. Additional Resources

Research Papers

- "LoRA: Low-Rank Adaptation of Large Language Models" (Hu et al., 2021)
- "Direct Preference Optimization: Your Language Model is Secretly a Reward Model" (Rafailov et al., 2023)
- "Training language models to follow instructions with human feedback" (Ouyang et al., 2022) — the foundational RLHF/InstructGPT paper

Related Modules

- Module 16 — LLM Evaluation & Testing (the rigor this module's Section 3.10 applies to fine-tuning decisions)
- Module 11 — Model Comparison (open-weight models most relevant to hands-on fine-tuning)
- Module 22 — Model Optimization (quantization, underlying QLoRA)

Communities & Further Learning

- Hugging Face's fine-tuning documentation and community for hands-on, framework-specific tutorials.